

- Communication Basics
- Microprocessor Interfacing
 - I/O Addressing
 - Interrupts
 - Direct Memory Access
- Arbitration
- Multilevel Bus Architectures
- Advanced Communication Principles
- Serial, Parallel and Wireless Protocols

5.1 Communication Basics

Basic terminology

Wires are the connecting lines of two terminals in communication system. It may be uni-directional or bi-directional. A single line can be used to represent multiples wires with the help of small angled line drawn through it.

Bus refers to the set of wires with a single function. Address bus for address, data bus for data are two examples of single functioned buses. Bus can also be the entire collection of wires. System bus, for instance, consists of address, data and control lines.

Port is the actual conducting device on periphery which connects bus to processor or memory or other devices. Port is a medium through which a signal is input to output from the processor. Port is also referred as pin which extends from the IC package and that can be plugged into a socket (IC base) on a printed circuit board. Metallic balls instead of pins may be present. However, metal pads are more common these days.

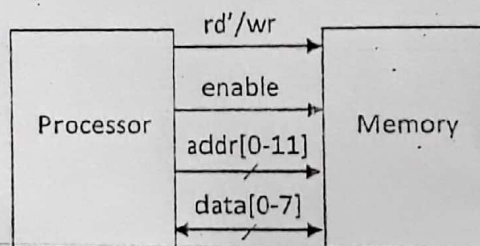


Figure 5.1: A simple bus example

Timing diagram is a diagrammatic representation for describing hardware protocol. In the diagram, time proceeds to the right along x-axis. It represents state of control lines or data lines. The control lines may be either low or high, whereas the data lines – address or data – can be valid or not valid. Active high means that a one on the line makes it active while active low means that a zero on the line makes it active. Asserting a line means making it active and de-asserting the line deactivates the line. A protocol may have several sub-protocols which are also called bus cycle or transaction. A bus cycle may consist of several clock cycles.

Example: Timing diagram for read protocol

The timing diagram of memory read protocol gives the following information to the designer

- The processor must set the **rd'/wr** line low for a read operation

- Address of memory must be placed on *addr* line for atleast t_{setup} time before setting the *enable* line high.
- Setting *enable* line high will cause memory to place the data on the *data* line after at time t_{read} .

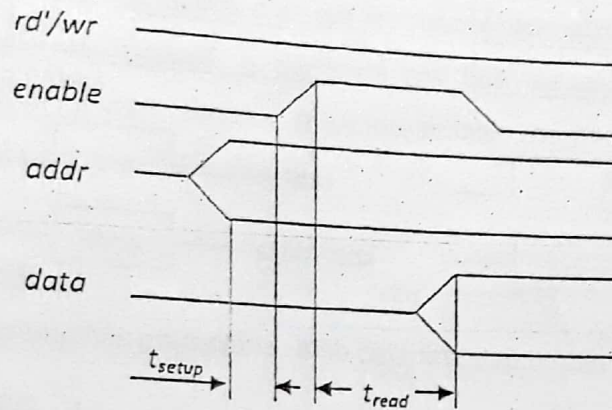


Figure 5.2: Timing Diagram: Memory read protocol

Basic Protocol Concepts

An actor is a device that can be processor or memory which takes part in data transfer. Actor can be a master or a slave. A master initiates the data transfer whereas a slave responds to the initiation request.

Data direction represents movement of data among actors. The direction of data is independent of type of actor. Either master or slave can send or/and receive data.

Addresses represent a special kind of data which specify a location in memory, a peripheral, or a register within a peripheral. A protocol often includes both an address and data. In every memory access protocol, the address specifies the location where the data should be read from or written to in the memory.

Time multiplexing represents a technique in which the multiple sets of data are sent one at a time over the shared line. Number of wires requirement can be reduced to a single line at the expense of time. The following figures show the examples of time multiplexing. In both cases, single bus is used to send multiple data at different time instant.

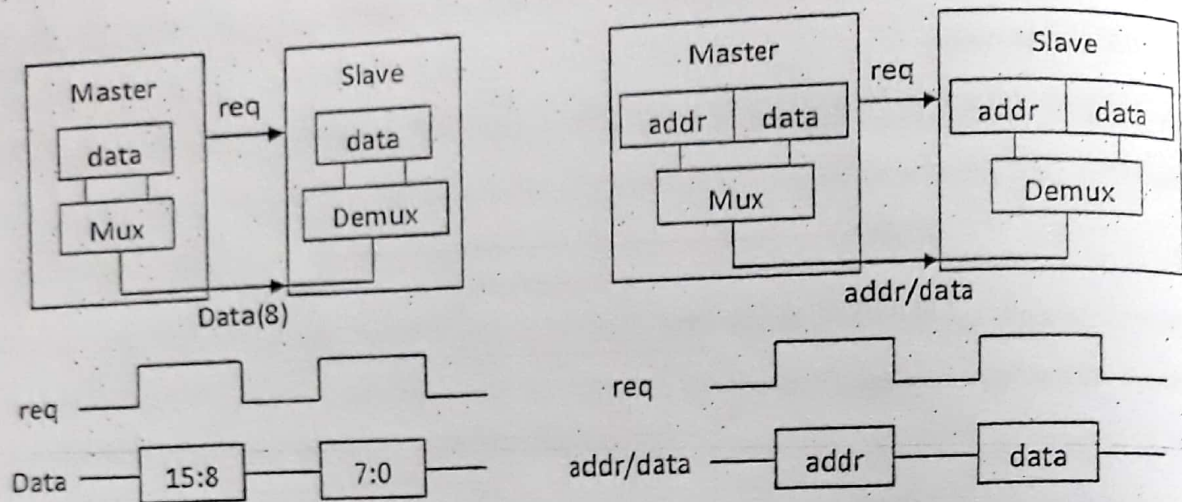


Figure 5.3: Time-multiplexed data transfer: data serialization and address/data muxing

Control methods are schemes for initiating and ending the data transfer. Strobe and handshake are two common control methods.

Strobe Protocol

In strobe protocol, master uses a control line and activates it to initiate the data transfer. Then the slave has certain time to put data on data bus. Assuming data to be valid, master reads the data from data bus and deactivates its control line. And both actors are ready for next data transfer. The main disadvantage of strobe protocol is that the master that initiates the transfer has no way of knowing whether the slave has received the data or not.

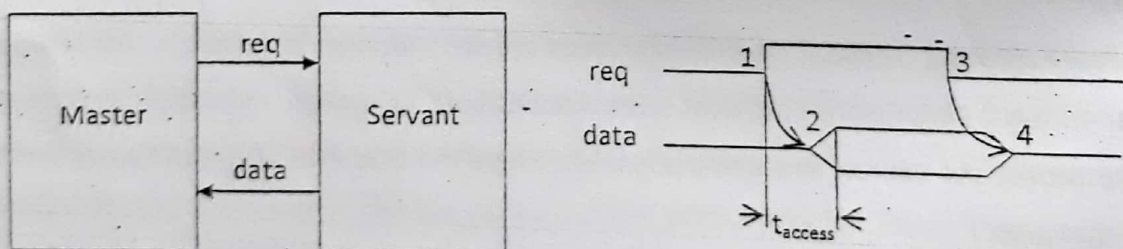


Figure 5.4: Strobe Protocol

The flow in timing diagram can be explained as:

1. Master asserts *req* to receive data
2. Slave puts data on *data* line within time t_{access}
3. Master receives data and deasserts *req*
4. Slave ready for next request

Handshake Protocol

In this protocol, servant uses extra line to acknowledge that the data is ready. Initially, master asserts request line to start the transfer. Then the servant, taking its time to put data on data line, asserts acknowledge line to inform the master that the data is ready. Next, the master reads the data from the data line and deasserts the request line which is followed by slave deasserting acknowledge line. Finally the transfer is complete and both actors are ready for next transfer. Though the protocol is somewhat complex, it is more reliable compared to strobe protocol as data availability is confirmed by the sending device.

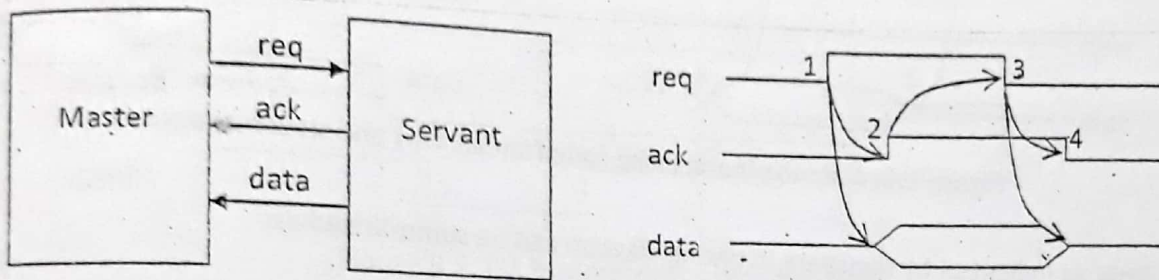


Figure 5.5: Handshake Protocol

The flow, as indicated by numbers, in timing diagram can be summarized as:

1. Master asserts *req* line to receive data
2. Servant puts data on *data* line and asserts *ack*
3. Master receives data from data bus and deasserts *req*
4. Servant ready for next transfer

Strobe/Handshake Compromise

A compromise protocol can be used to achieve the speed of strobe protocol and varying response time tolerance of handshake protocol. As represented in figure 5.6, servant can use wait line, if it is not ready to put data on data line.

- If the servant can put data within time t_{access} then it follows strobe protocol representing fast response. And wait line remains unused in this protocol.
- If the servant can't put the data within t_{access} time then it asks master to wait longer by asserting wait line. After the data is ready, the wait line is deasserted by servant and master receives the data. And it represents slow response as master has to wait for certain time for the data transfer

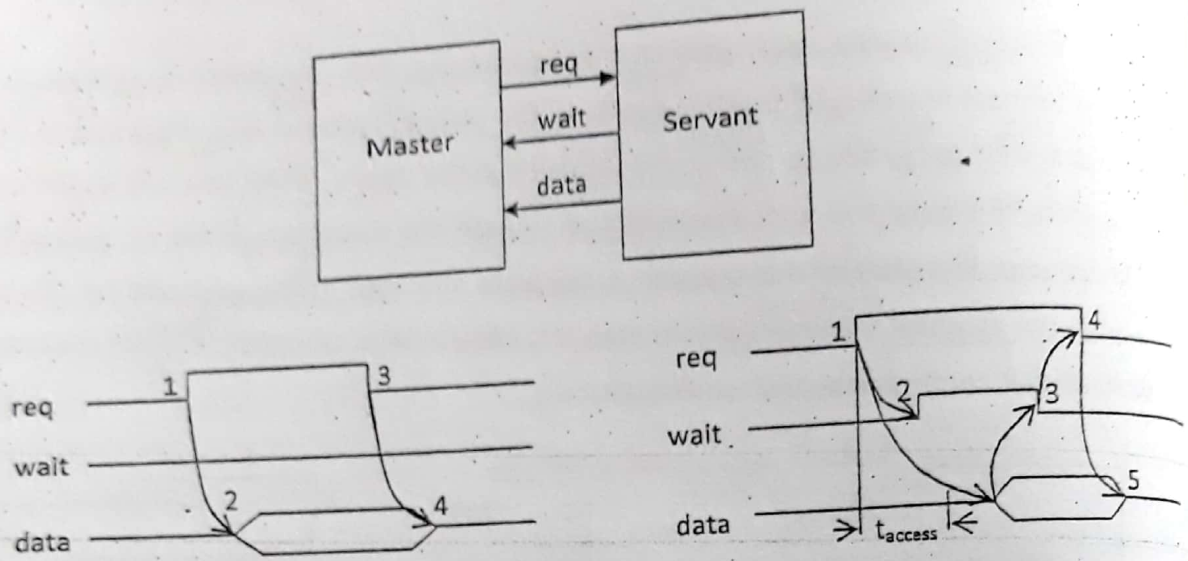


Figure 5.6: A strobe/handshake compromise: fast and slow response

The flow as indicated by numbers in timing diagram can be summarized as:

For fast response

1. Master asserts *req* line to receive data
2. Servant puts data on data bus within time t_{access} , *wait* line remains unused
3. Master receives data and deasserts *req*
4. Servant ready for next request

For slow response

1. Master asserts *req* to receive data
2. Servant can't put data within t_{access} , asserts *wait* line
3. Servant puts data on bus and deasserts *wait*
4. Master receives data and deasserts *req*
5. Servant ready for next request

Example: The ISA Bus Protocol – Memory Access

The Industry Standard Architecture bus protocol is common in systems using an 80x86 microprocessors. The processor uses 20-bit memory address and follows compromise strobe/handshake protocol. If the memory is not ready then the processor inserts wait-cycles. Four cycles is default for the operation to complete. For the read operation, in the first clock cycle the processor puts address on the address line and asserts address latch enable signal. During second and third clock cycle, the processor asserts memory read signal. After third clock cycle, the data is

available on data lines. Finally all signals are deasserted at fourth clock cycle. The timing diagram for memory read operation and memory write operation is shown in the figure below.

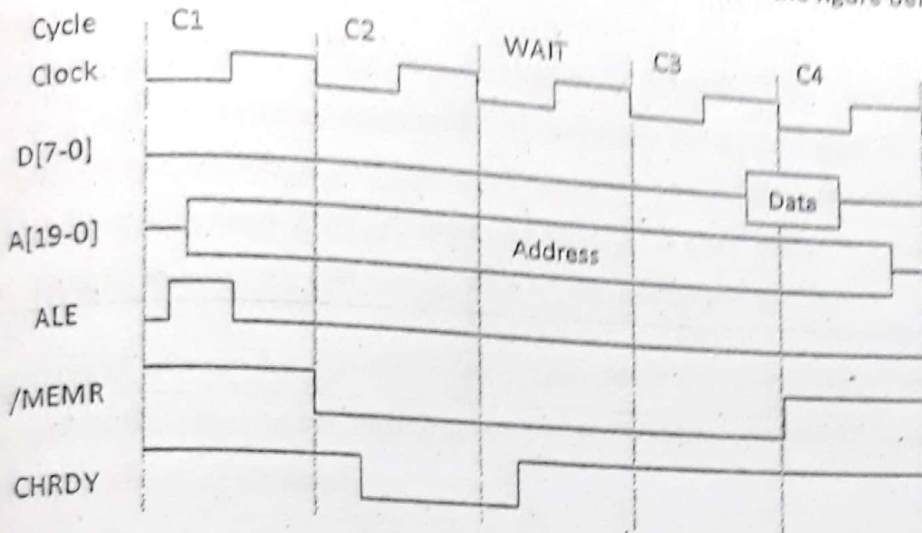


Figure 5.7: ISA bus protocol – read bus cycle

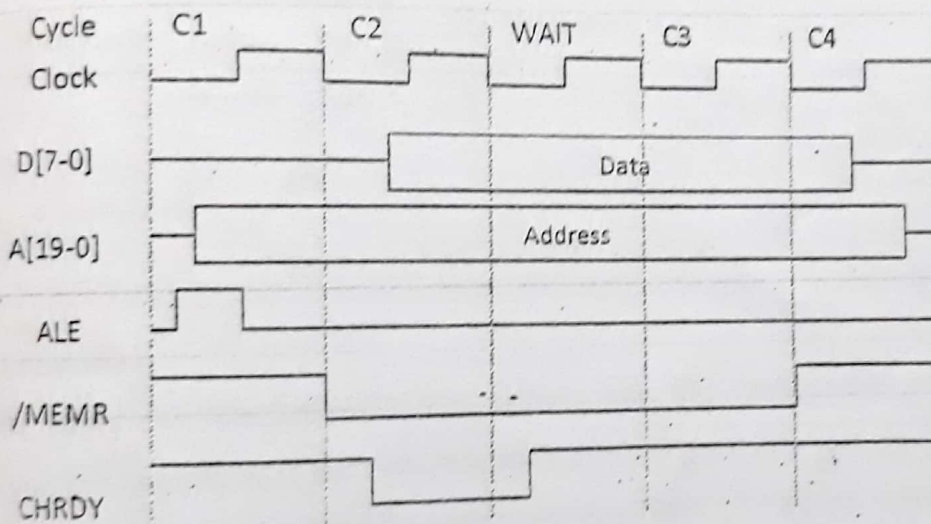
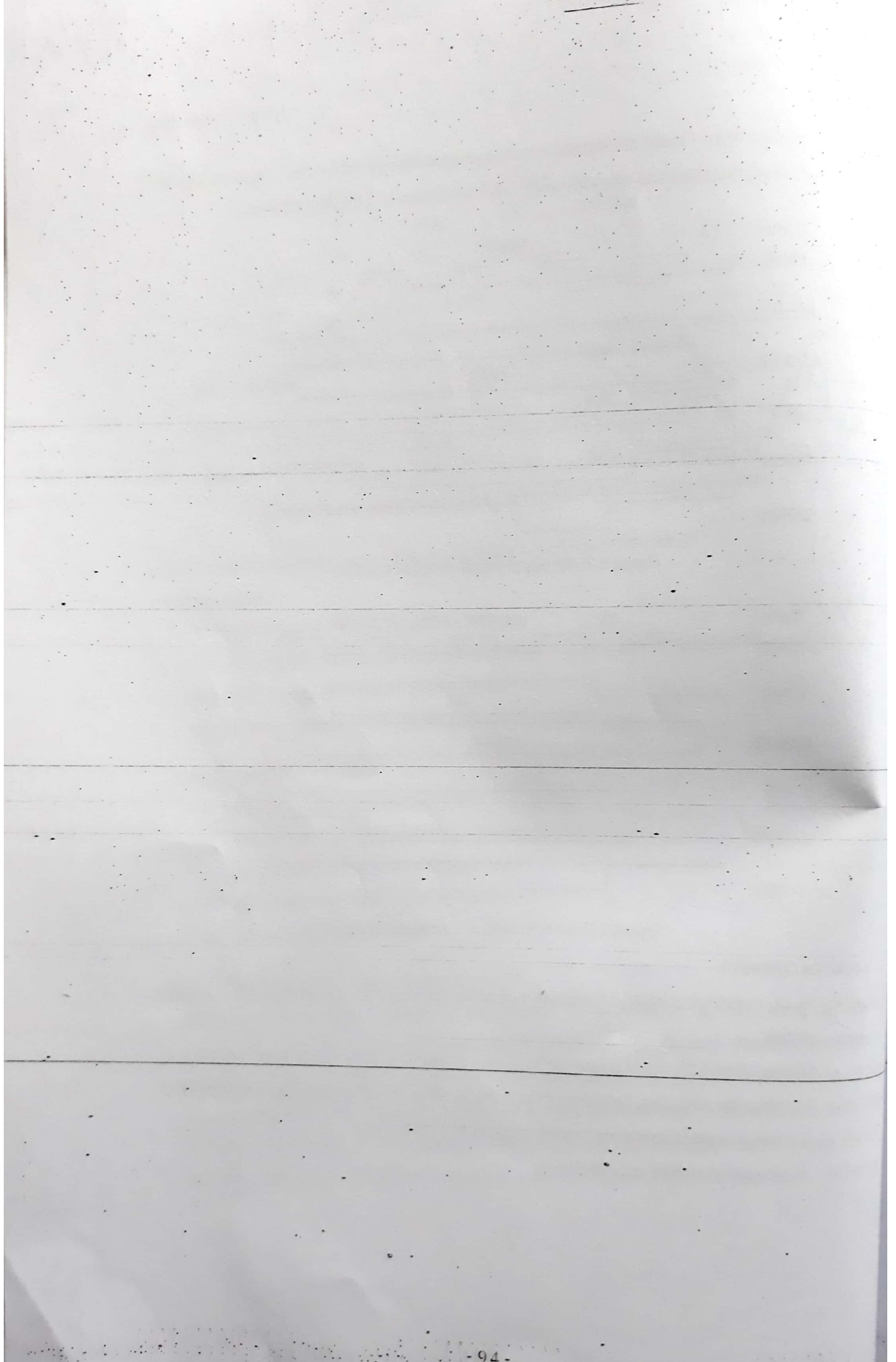


Figure 5.8: ISA bus protocol – write bus cycle

for Write Operation

- In C1, processor puts 20 bit address memory address on the address line and asserts ALE signal.
- During C2 and C3, the processor puts the data on the data line and asserts MEMW signal to enable write operation. However, if the memory, when not ready, deasserts CHRDY signal in C2 then processor inserts wait cycles until CHRDY is reasserted.
- In cycle C4, all signals are deasserted.



5.2 Microprocessor Interfacing

A. I/O Addressing

port based I/O

In port based I/O, a port can be directly read from or written into with the help of processor instructions. It is also referred as parallel I/O. Generally the devices may be provided with one or more N-bit ports to facilitate port based I/O and each port is bit addressable. For example, 8051, AVR microcontrollers have 8 bit I/O ports. In 8051, $P1 = 0xF7$ statement will write into Port 1 of microcontroller. Also, for bit addressing, $P2.4 = 1$ will set the pin 4 of Port 2.

The port based I/O can be extended using appropriate peripheral which extends the number of available ports from four to six. Each port on peripheral is associated with a register that can be read or written into by the processor,

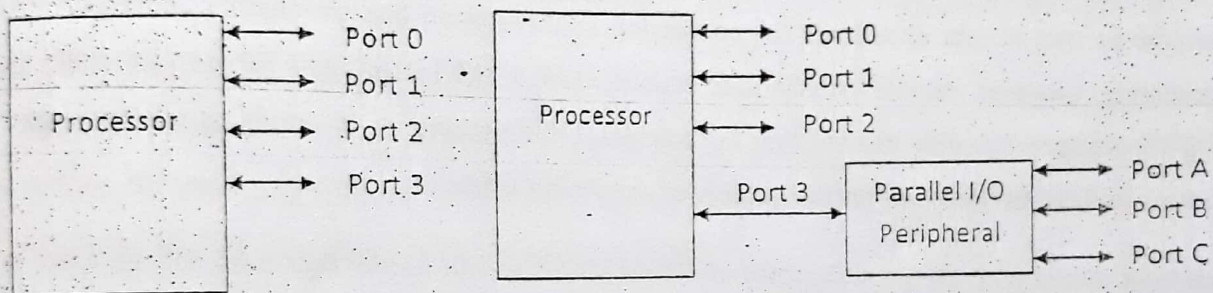


Figure 5.9: Port Based I/O and Extended Parallel I/O

Bus based I/O

In bus based I/O, the processor has address, data and control lines for I/O addressing. The communication protocol is built into the processor. A single instruction is available which causes the hardware to write or read data. If a system with bus based I/O requires parallel I/O then parallel I/O peripheral can be connected to the system bus.

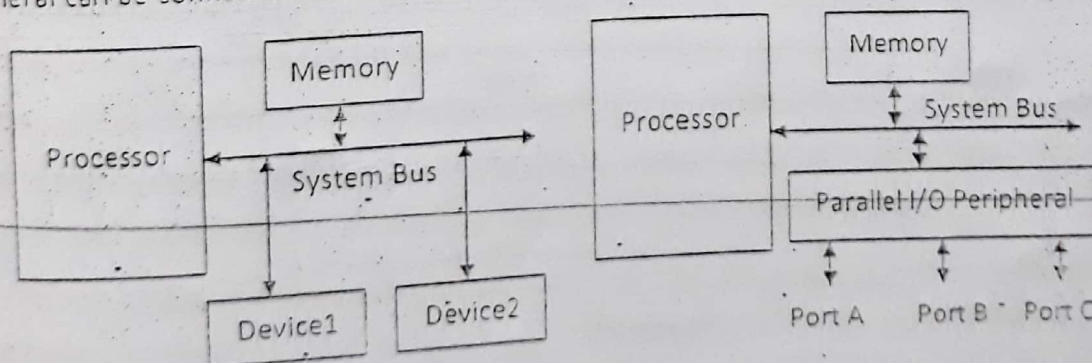


Figure 5.10: Bus-based I/O and Extended bus-based I/O with parallel I/O peripheral

Memory-Mapped I/O

Memory-mapped I/O is a type of bus-based I/O addressing for a processor to communicate with peripherals in which peripherals are addressed using the specific existing address space. The total address space is divided into memory address and peripheral address. Hence, there is loss of memory addresses to peripherals. Also, no special instructions for peripherals are required for data transfer, since instructions like MOV used for memory will also work with peripherals.

Example: A bus with 16 bit have total of 65536 addresses. So, lower 32768 addresses may correspond to memory address while upper 32768 correspond to I/O addresses.

Standard I/O

Standard I/O is a type of bus-based I/O addressing in which extra control line (M/I/O) is used to indicate whether the address represents memory location or peripheral. Memory locations and peripherals use all sets of address for addressing, so there is no loss of memory addresses. This addressing, however, requires special instructions. MOV, LOAD instructions for memory while IN, OUT for peripherals. Also the address decoding logic for peripherals is simple as the high order address bits can be ignore when the number of peripherals is less.

Example: A bus with 16-bit have total of 65536 addresses. All 65536 addresses can be used to address memory and peripheral. The M/I/O line is used to select either memory or peripheral. If M/I/O is zero then the address on the address bus corresponds to a memory address.

Example: The ISA Bus Protocol – Standard I/O

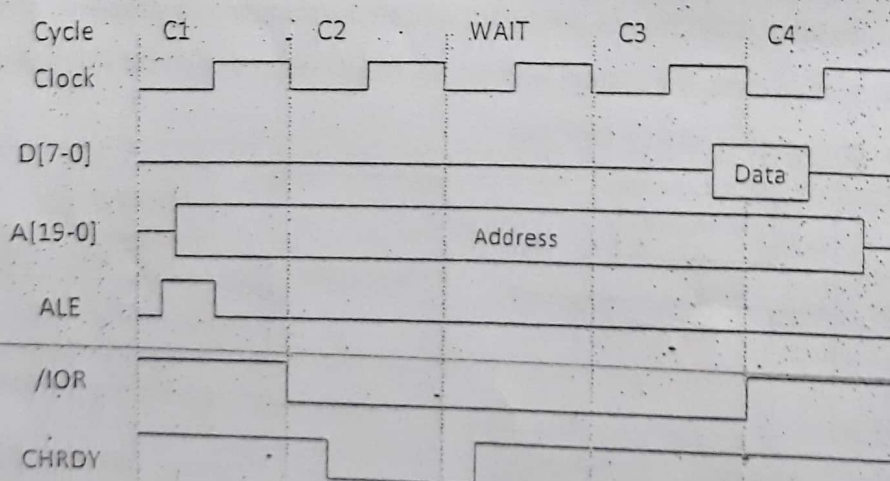


Figure 5.11: ISA bus protocol for standard I/O

Example: A Basic Memory Protocol

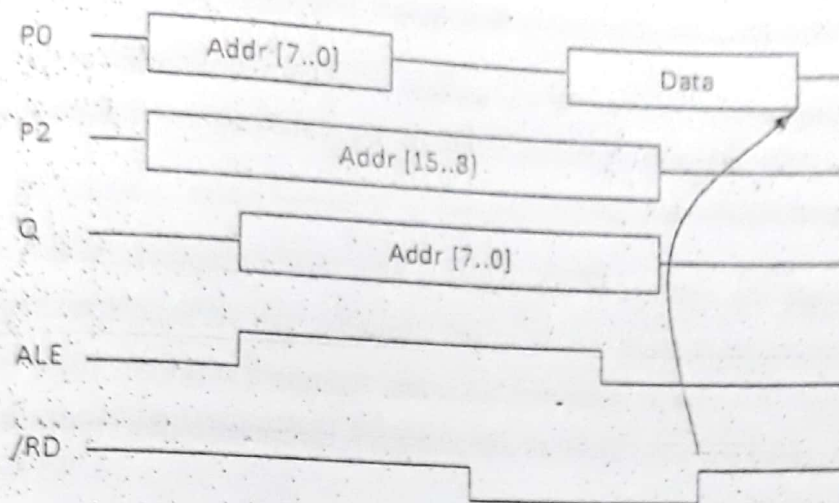


Figure 5.12: A Basic Memory Protocol for 8051 microcontroller: Timing diagram for read operation.

Read Operation in 8051 microcontroller

- Microcontroller places source address on ports P2 and P0. Port 2 holds 8 most significant address bits and retains its value throughout the read operation. Port 0 holds the eight least-significant address bits which is stored using a 8-bit latch.
- The ALE signal is used to trigger the latching of port P0. And controller asserts high impedance on P0 to allow memory device to drive it with requested data. The memory outputs valid data as long as RD signal is asserted. The microcontroller reads the data and deasserts its control port signals.

B. Interrupts – Interrupt Driven I/O

The peripherals may require service from the processor which is very much unpredictable. So there is an issue on how to serve the peripherals by the processor as it remains busy on its own task. Polling and interrupts are two basic methods to address that issue.

1. Polling is a method in which the processor checks for service requirement of every peripheral. This method, though, is easier and simple to implement, the repeated checking, however, wastes many clock cycles which could have been used to do certain useful work.
2. Interrupt is a feature of the processor through which the peripherals can request for service even when processor is busy in its own task. For external interrupt, there is always a pin available to implement interrupt feature. Whenever the interrupt pin is asserted, the processor jump to a particular address at which the routine for the interrupt is stored. Interrupt

overcomes the limitations of polling, but interrupt, in itself, is the type of polling. The pin is checked after the execution of every instruction, so it does not require extra clock cycles.

Interrupt address vector represents the address in which the interrupt service routine (ISR) resides. Fixed interrupt and vectored interrupt are two common methods by which the processor obtains the address of ISR.

In fixed interrupt, the address of subroutine is built into microprocessor and remains fixed. Programmer simply has to store the ISR at that location or can put jump instructions to move to actual location of ISR where programmer has saved. Suppose a data from sensor (peripheral1) is to be read, processed and then a motor (peripheral2) is controlled based on calculated data. The flow of actions can be summarized as:

- Peripheral1 has data in its register; meanwhile the processor is executing its main program.
- Peripheral1 asserts INT to request service from the processor.
- After execution of each instruction, the processor checks INT pin. So processor detects the service requirement. It saves its present context and sets the PC to the fixed ISR location.
- The ISR is executed which reads data from peripheral1, modifies it and sends the resulting data to peripheral2. At the same time, peripheral1 deasserts INT after data is read from it.
- The processor retrieves its state and resumes its work.

In vectored interrupt, peripheral must provide the address to the processor. In this method, along with INT pin, INTA pin is also required to acknowledge that the interrupt has been detected and the peripheral can provide the address of relevant ISR using system bus. The peripheral provides the address through the data bus which is read by microprocessor.

The flow of actions can be summarized as:

- Peripheral1 has data in its register; at the same time the processor is executing its main program.
- Peripheral1 asserts INT to request service from the processor.
- After execution of each instruction, the processor checks INT pin. So processor detects the service requirement and it asserts INTA.
- Peripheral1 detects INTA and puts interrupt address vector on the data bus.

- Processor jumps to the address read from data bus and executes its corresponding ISR. It reads data from peripheral1 processes it and sends the result to the peripheral2. Meanwhile peripheral1 deasserts INT after data is read from it.
- The processor retrieves its state and resumes its work.

In interrupt address table, which is a compromise between fixed and vectored interrupt methods, a table with ISR addresses is stored in memory of processor. A peripheral instead provides the number, rather than the address of ISR, corresponding to an entry in the table. One major advantage is that the bit requirement to address the table is very less compared to number of bits of real address of ISR. Also it provides the flexibility to assign and change the location of ISR.

Additional Interrupt Issues

External interrupts may be maskable or nonmaskable. In maskable interrupt, the programmer can use specific instruction to disable the interrupt by configuring certain bits of interrupt register. It is important when more critical works need to be executed first. Nonmaskable interrupt cannot be disabled by the programmer. It requires a separate pin for drastic situations. For instance, if power fails, the nonmaskable interrupt can cause a jump to a subroutine that stores critical data in non-volatile memory, before power is completely gone.

Another issue regarding the interrupt is jump to ISR in which the microprocessor either saves complete context or partial state before jumping to ISR. Some processors save PC, registers which consumes many cycles, while others save the content of PC only. The ISR, however, must not modify registers if its content is not saved.

C. Direct Memory Access – DMA controller.

INTRODUCTION

When the communication between memory and peripherals involves microprocessor then there will, somehow, always be waste of processor's time. Since the speed of the processor and peripherals may not match, data must be stored temporarily before processing which is referred as buffering. Buffering will, certainly, impact on system performance. Also while using interrupt feature, the storing and restoring of state of processor is an inefficient process, since this process requires many clock cycles. And, the regular program stalls during transfer of data causing more problems in the performance of the system. So, a separate single-purpose processor called a DMA

controller is required which relieves processor from all data transfers involving memory and peripherals.

DMA controller is specifically used to transfer data between memories and peripherals. The peripherals request the service from DMA controller which then requests control of the system bus from processor. After that, processor relinquishes the system bus. Finally, the data transfer between memory and peripheral is initiated by DMA controller without the involvement of processor. Hence, the overhead required for storing and restoring the state is eliminated. Also, the processor can continue its regular task unless it requires the system bus or the particular data being transferred.

BLOCK DIAGRAM

The simple block diagram of system involving DMA controller is shown in the figure below:

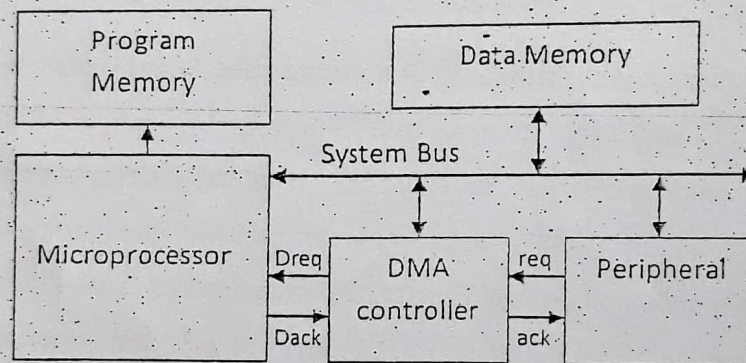


Figure 5.13: Simple system with DMA Controller

OPERATION

The flow of action for the transfer of data between peripheral and memory using DMA can be summarized as:

- Initially, processor is busy executing its main program.
- After peripheral has data within its register it asserts request line for service from DMA.
- DMA asserts request signal to request the system bus from processor.
- Processor releases the system bus after seeing the request from DMA, and acknowledges about it to DMA.
- DMA asserts acknowledge signal to peripheral, and starts transfer of data as requested.
- After the completion of transfer, all control lines are deasserted and processor retakes the control of the system bus.

5.3 Arbitration

Arbitration is the mechanism through which a service or shared resource is provided to particular requesting device, out of many contenting devices for service.

A. Priority Arbiter

INTRODUCTION

Priority arbiter is a single purpose processor which is used to arbitrate among various requests from peripherals. Each of the peripherals, which are connected to the arbiter, can make request for the service. Using certain priority mechanism, arbiter selects a peripheral to permit the required service. The figure 5.14 shows the priority arbiter connected with peripherals which use vectored interrupt to request service and processor which provide service to the peripherals. Arbiter is connected to system bus for configurations only. The configurations may include setting priorities of the peripherals.

The main advantage of this arbitration is that it can support advanced priority schemes. Also, failure of single peripheral does not have any impact on the operation of whole system. The system, however, must be redesigned if new peripherals are to be added. So, this method is less flexible if new peripherals are required to be added or removed.

BLOCK DIAGRAM

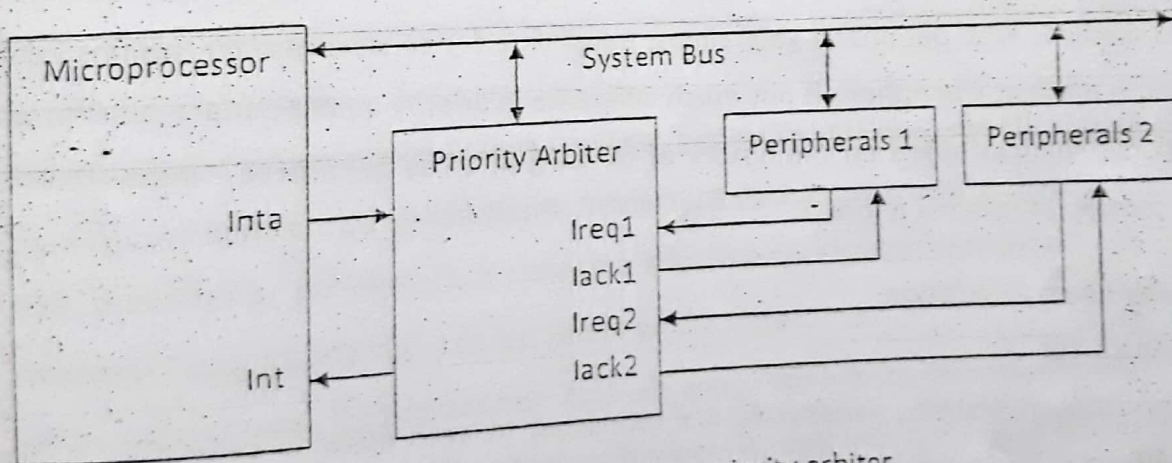


Figure 5.14: Arbitration using a priority arbiter

OPERATION

The stepwise operation of arbitration using priority arbiter is listed below:

- Initially, microprocessor is busy in its own operation.

- Both peripherals can assert request to priority arbiter which interrupts processor when at least one request is available from peripherals.
- Processor stops its current operation, stores its state and asserts interrupt acknowledgement signal.
- After acknowledged by processor, priority arbiter asserts acknowledge signal to any one peripheral based on priority.
- The selected peripheral puts its interrupt address vector on the system bus.
- Microprocessor reads ISR address from data bus and jumps into it, executes the ISR.
- After execution of requested ISR, processor retrieves its state and resumes its operation.

TYPES OF PRIORITY ARBITER

The priority among peripherals can be determined based on, basically, two schemes; fixed priority or rotating priority.

Fixed Priority

Each peripheral is assigned a unique rank. If two peripherals simultaneously request for service then the arbiter chooses the one with the higher rank. Such method is efficient when there is a clear distinction in priority among peripherals. But it can cause high-ranked peripherals to get much more servicing than other peripherals.

Rotating priority or Round – robin priority

In this method, each peripheral gets almost equal time for service from the arbiter. This priority method is efficient when there is not much difference in priority among peripherals. The priority of peripherals changes based on the history of servicing of those peripherals, so the arbiter can get more complex in rotating priority.

B. Daisy-Chain Arbitration

INTRODUCTION

In daisy-chain arbitration, peripherals are connected to each other in daisy-chain manner. The arbitration is built within the peripherals with each having a request and acknowledge signals as shown in the figure 5.15. The request signal and acknowledge signals flow through the peripherals: peripheral's request signal flows downstream to processor and processor's acknowledge signal flows upstream to requesting peripheral. The peripheral connected first to the processor has the highest priority while the peripheral at the end of chain has lowest priority.

The main advantage of this arbitration method is that one can easily add or remove peripherals from the system without the requirement of system redesign. This method, however, does not support rotating priority. Also, if one peripheral is damaged in the chain, other peripherals beyond that broken point will remain inaccessible as signal cannot pass through the chain.

BLOCK DIAGRAM

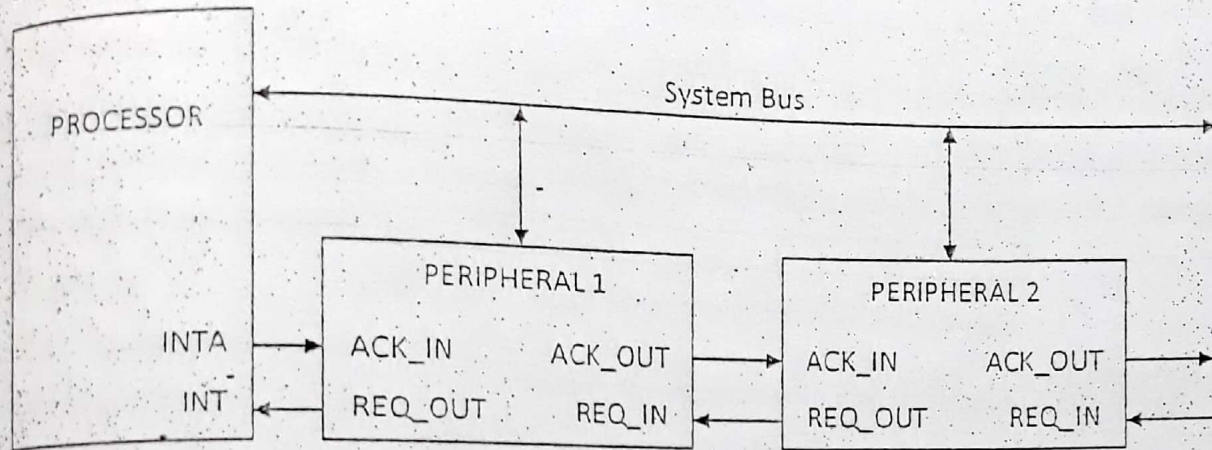


Figure 5.15: Daisy Chain Configuration

OPERATION

Suppose peripheral 2 requires service from the processor then the operation can be summarized as:

- Microprocessor is busy in executing its own program.
- The request signal from peripheral 2 is sent to processor through the peripheral 1 and interrupt pin is asserted.
- Processor stops its current work, stores its state, and asserts acknowledgement signal.
- The acknowledgement signal reaches to peripheral 2 through peripheral 1. Since the request is not generated by peripheral 1, it passes the acknowledge signal to peripheral 2.
- Peripheral 2 puts its interrupt address vector on the system bus.
- Microprocessor reads ISR address from data bus and jumps into it, executes the ISR.
- After execution of requested ISR, processor retrieves its state and resumes its operation.

Daisy Chain aware peripherals

Generally, peripherals have acknowledge input and request out lines-but daisy chain aware peripherals must have additional acknowledge output and request input lines then they will not be daisy chain aware peripherals. But they can be made daisy chain aware by certain logic whose complexity

may increase based on complexity of system. One simple example for making a peripheral daisy chain aware is shown in the figure below.

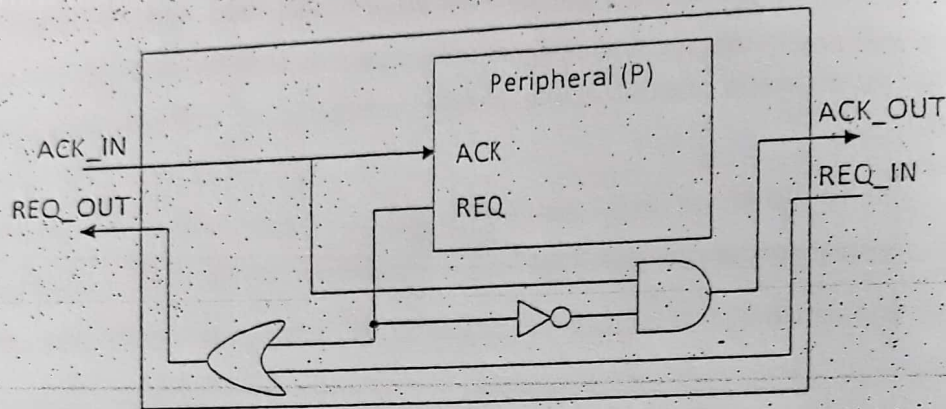


Figure 5.16: Simple Logic to make Daisy Chain Aware

Case 1: When request is from downstream peripherals

- Peripheral (P) does not participate in the flow of signal

Case 2: When request is from upstream peripherals beyond peripheral (P)

- $REQ_IN = 1$ but $REQ = 0$, resulting in $REQ_OUT = 1$
- $ACK_IN = 1$ and $REQ = 0$, resulting in $ACK_OUT = 1$

Case 3: When request is from peripheral (P)

- $REQ = 1$, $REQ_IN = X$ (don't care), resulting in $REQ_OUT = 1$
- $ACK_IN = 1$ and $REQ = 1$ resulting in $ACK = 1$ and $ACK_OUT = 0$

C. Network-Oriented Arbitration

In network oriented arbitration, arbitration is done for multiple microprocessors sharing a common to form a network. Arbitration is build into the bus protocol, as bus is the only the medium that connects multiple processors. However, multiple processors may try to access the bus simultaneously resulting in data collision. The protocol must be designed in such a way that the contending processors don't start sending the data at the same time. Also some statistical methods can be used so as to make chances of data collision very rare, if not eliminate it. Some protocols use efficient address encoding schemes in which higher priority address will override the lower-priority one.

5.4 Multilevel Bus Architectures

Multilevel bus architectures are implemented in the system to improve the overall performance of the system. One can easily presume a single high-speed bus would be enough for all the communications in the system. But, however, there are various drawbacks of using single high speed bus. Few of them are discussed in the following paragraph.

Inefficient interface
For a single high speed bus, each peripheral requires a high-speed bus interface. But the peripherals may not need such high-speed transfer resulting in extra power consumption, increase in number of gates, and high cost. Also, the high-speed bus can be very processor specific which can lead the interface of a peripheral to be non portable.

Slower bus

When many peripherals are connected to a single bus, all peripherals may not get the access to the bus when required. This condition results to slow down the speed of transfer, hence it can create a performance lag.

Two Level Bus Systems

Generally, two level bus systems consist of a high-speed processor local bus, a lower-speed peripheral bus and a bridge to connect two buses.

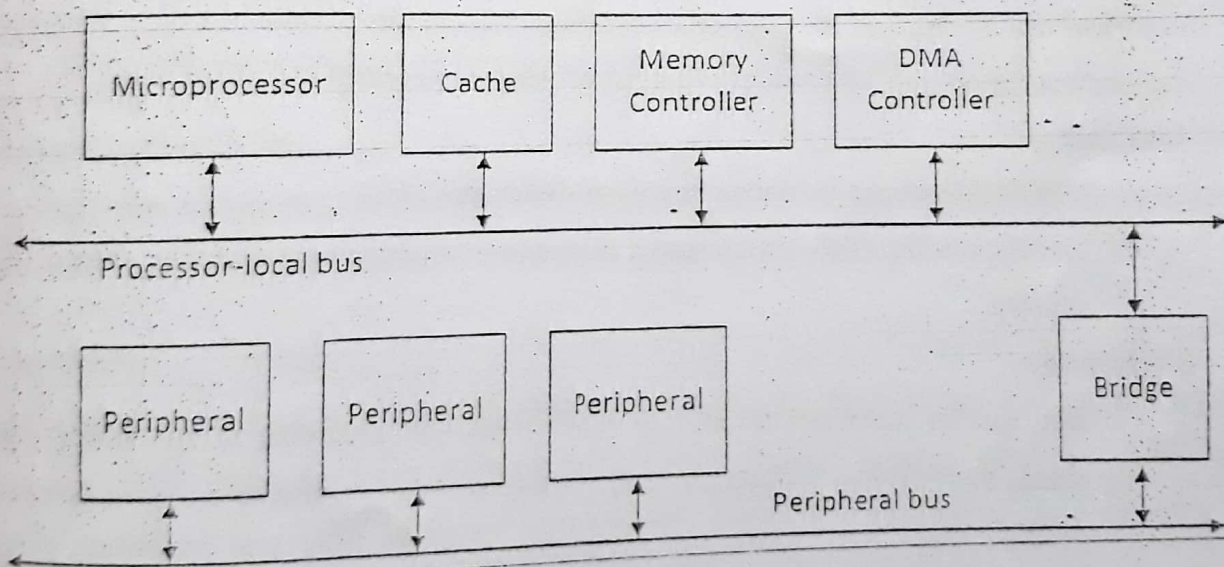


Figure 5.17: A two-level bus architecture

The processor local bus connects very high speed devices such as microprocessor, cache, memory controllers, and certain high-speed coprocessors. These buses are wide, as wide as a memory word and frequent communication takes place through it.

- On the other hand, the peripheral bus connects to those peripherals which do not have access to processor-local bus. It emphasizes on portability, low power, or low gate count. It is often narrower and slower than a processor local bus. The frequency of communication through peripheral bus is also less as compared to that of processor local bus. So the interface for peripheral bus is comparatively efficient one in terms of number of pins, gates and power consumption.
- A bridge is a single purpose processor that connects the two buses of the system and also makes the various conversions required. Speed synchronization is another important function of bridge. Data speed and data formats of processor-local bus is different to that of peripheral bus, such problem is resolved by bridge using various mechanisms.

Three-level bus hierarchy

Three-level bus systems consist of processor local bus, system bus and peripheral bus. A local bus connects the processor to a cache and may support one or more local devices. The system bus, acting as high-speed bus, offloads much of the traffic from the processor local bus. And the peripheral bus is used to connect various peripherals in the system.

5.5 Advanced Communication Principles

Parallel Communication

In parallel communication, the physical layer carries multiple bits of data at a time. With each wire carrying a single bit, the bus consists of data wires along with control and power lines.

Advantages

- High data throughput: Many bits are transferred at a time.
- Less complexity: Easily implemented in hardware requiring only a latch to copy data onto a data bus.

Disadvantages

- Long parallel wires can result in Ferranti Effect. And according to this effect, there is a voltage build up due to capacitance and voltage at receiving end becomes more than that of sending end.
- Little variation in wire length can cause data misalignment as the bits at the receiving ends with reach at different time.
- It is more costly to construct and can make system bulky. The cost further increases if insulation of wires to prevent the interference is considered.

Usage

- It is used to connect devices which reside on the same circuit board or same IC.

Serial Communication

In serial communication, the physical layer carries one bit of data at a time. With all bits of data passing through the single wire, the bus is composed of single data wire along with control and power lines.

Advantages

- Significant reduction in the size, the complexity of the connectors and the associated costs.
- Throughput can be better for two distant devices as compared to parallel communication of two distant devices.
- It does not exhibit Ferranti effect and data misalignments.

Disadvantages

- Complex interfacing logic and communication protocols; the data are decomposed into bits at sending end, which must be assembled properly at receiving end.
- For short distance communication, its throughput is very less as compared to that of parallel communication.

Usage

- It is used to connect distant devices. But it doesn't mean that it cannot be used to connect devices at short distance. However, it is more efficient for distant communications.

Wireless Communication

In wireless communication, the devices do not need to be connected physically for data transfer. Infrared and radio frequency channels are used as a physical layer.

Infrared wave

Infrared wave, which cannot be seen by naked eye, uses electromagnetic wave frequencies that are below the visible light spectrum. Infrared waves are generated using infrared diode whereas infrared transistors are used to detect the infrared emitted by infrared diode. Such infrared transistors conduct when exposed to infrared wave. One advantage of infrared communication is that it is cheap to build transmitters and receivers. But the main disadvantage of this sort of communication is that it requires line of sight between the two devices participating in communication.

communication. Also the range of communication is low, which makes it an inefficient method of communication for distant devices.

Radio Frequency

Radio frequency uses electromagnetic wave frequencies in the radio spectrum. For such communication, analog circuitry as well as an antenna is required at communicating devices. The main advantage of this type of communication is that the line of sight is not required. Also the longer distance communication is possible. The range of communication is dependent on the transmission power. But building transmitters and receivers can be complex and costly in Radio Frequency communication.

Layering

Layering the communications process means breaking down the communication process into smaller and easier to handle interdependent categories, with each solving an important and somehow distinct aspect of data exchange process. Layering can also be viewed as a hierarchical organization of a communication protocol where lower levels of the protocol provide services to the higher levels. The main objective is to break the complexity of a communication protocol into simple levels which ensures easier handling and simplified design. The physical layer provides the lower level services of sending and receiving bits or words of data, whereas the application layer provides the high level service to the user.

Error Detection and Correction

Error detection is the process of detecting errors that may occur during the transmission of data in any communication process. Errors can be bit error or burst of bit errors. In bit error, single bit in the transmitted data is invalid. But in case of burst of bit errors, more than one bit gets changed.

Error correction is the process of correcting the bits that were detected during communication process. Parity and checksum are two basic error correction methods.

In parity check, extra bit is send along with data to provide additional information about the data. If extra bit makes an odd number of 1s in data word bits plus parity bit then it is referred as odd parity otherwise it will be even parity. The parity of data must be check at both ends of communication and the parity of the data sent must be same to that of parity of data received. This type of checking

method is efficient for single bit error but can create problems for burst of bit errors as it is not able to detect change in even number of bits.

Example:

Data of 7 bits – 0011010

Transmitted data with even parity – 00110101

Received data with parity – 10110101, it indicates error as it should have an even parity.

Received data with parity – 10010101, change in two bits and error not detected.

In checksum error checking, multiple words of data in packets are checked for error. The extra word which represents the XOR sum of all data words in a packet is transmitted along with packet. Though it can be implemented for burst of bits error, it does not account for all error combinations. The transmitter sends the packet of data along with the checksum word which is checked at receiving end on reception. If the checksum word is correct then it represents successful transmission. However, few error combinations can generate the checksum word same as received, in such case the error checking fails.

Example:

Data words of packet to be transmitted: 010101, 011101, 110011, 101100

Checksum word at transmitter: 010111 (XOR of all data words)

Received checksum word: 010111

Received data words of packet: 110101, 010101, 110011, 101100, error exists

Calculate Checksum word at receiver: 111111, checksum does not match, error check success.

Received data words of packet: 010101, 011101, 110011, 101100, error exists

Calculate Checksum word at receiver: 010111, checksum match, error check fails.

5.6 Serial, Parallel and Wireless Protocols

A. Serial Protocol

Inter-IC or I²C or I²C

I²C is a serial protocol for two-wire interface to connect low-speed devices like microcontrollers, EEPROMs, A/D and D/A converters, I/O interfaces and other similar peripherals in embedded systems. The I²C has 7-bit or 10 bit address space. Seven bit addressing allows a total of 128 devices to communication over a shared I²C bus. The common speed of I²C bus is 100 Kbit/s in standard

mode and 10 Kbit/s in low-speed mode. Recent revisions of I2C can host more nodes and run at faster speeds: 400 Kbit/s in Fast mode, 1 Mbit/s in Fast mode plus and 3.4 Mbit/s in High speed mode. I2C uses only two wires: SCL (serial clock) and SDA (serial data). Length of the wires is not limited as long as the total bus capacitance is less than 400pf.

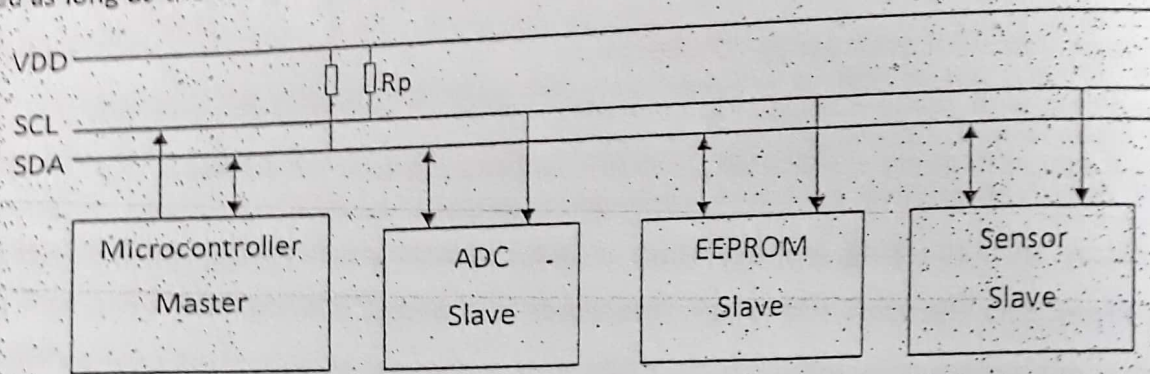


Figure 5.18: I2C bus structure

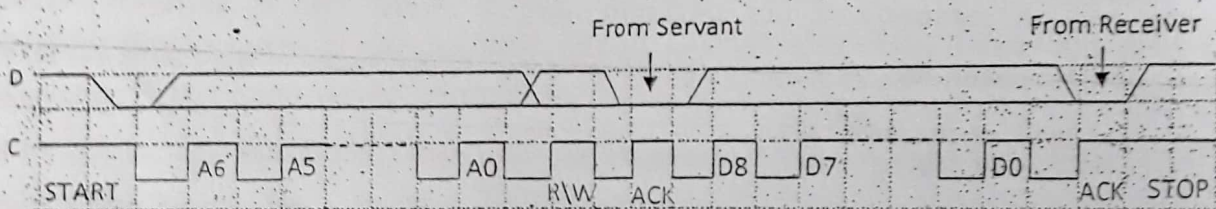


Figure 5.19: Timing diagram of a typical read/write cycle

A typical I2C byte write cycle operates as follows:

- The master initiates the transfer with a start condition. Start condition is represented by a high to low transition of SDA line while the SCL is held high.
- Then, the address of the device to which the data is to be written is sent with most significant bit down to the least significant bit.
- For write operation, the master sends a zero after sending the address. And the slave acknowledges the transmission by holding the SDA line low during first ACK clock cycle.
- Next, the master transmits a byte of data with most significant bit first.
- The slave acknowledges the reception of data by holding the SDA line low during second ACK clock cycle.
- Finally, master terminates the transfer by generating a stop condition. Stop condition is represented by a low to high transition of SDA line while the SCL is held high.

Serial Peripheral Interface (SPI)

The serial peripheral Interface bus is a synchronous serial communication interface specification used for short distance communication. It is used to send data between processors/controllers and small peripherals. It uses separate clock and data lines along with a select line to choose the device that should be communicated with. The SPI bus consists of four logic signals: SCLK – serial clock, MOSI – Master Output Slave Input, MISO – Master Input Slave Output, and SS – Slave select. The SPI bus can operate with a single master device and with one or more slave devices. Full duplex communication, higher throughput, simple software and hardware implementation, etc are few characteristics of SPI protocol.

Control Area Network (CAN)

CAN is an International Standardization Organization (ISO) defined serial communications bus originally developed for the automotive industry to replace the complex wiring harness with a two wire bus. The specification calls for high immunity to electrical interference and the ability to self-diagnose and repair data errors. These features have led to CAN's popularity in a variety of industries including building automation, medical, and manufacturing.

Some of the characteristics of the CAN protocol includes high-integrity serial data communications, real-time support, data rates up to 1 Mbit/s, error detection and confinement capabilities. Balanced differential signaling in CAN protocol not only reduces noise coupling but also allows high signaling rates over twisted pair cable. The CAN protocol incorporates five methods of error checking which forces transmitting node to resent the message until it is received correctly. But if the error limit is reached then the faulty node is deprived of transmit capability. Faulty nodes are automatically dropped from the bus, which prevents any single node from bringing a network down. This error containment also allows nodes to be added to a bus while the system is in operation, otherwise known as hot-plugging. It implements a non destructive, bit-wise arbitration in which the node winning arbitration continues with the message without being corrupted by another node. The high speed ISO 11898 standard specifications are given for a maximum signaling rate of 1 Mbps with a bus length of 40m with a maximum of 30 nodes.

To summarize, the protocol defines data packet format and transmission rules to prioritize messages, guarantee latency times, allow for multiple masters, handles transmission errors,

retransmit corrupted messages, and distinguish between a permanent failures of a node versus temporary errors

FireWire

FireWire is a serial bus protocol for high-speed data transfer. It was initiated by Apple and developed by IEEE P1394 group so may refer this protocol as IEEE 1394. It supports mass information transfer and allows peer to peer device communication without the involvement of system memory or CPU. Some of the characteristics of FireWire protocol include transfer rates up to 400 Mbit/s, plug and play and hot swapping, packet based layered design structure and provision of power through the cable. Also, the 64 bit addressing allows a local-area network to consist of 1023 sub-networks, each consisting of 63 nodes. FireWire devices are organized at the bus in a tree or daisy chain topology. In arbitration, the closest node requesting for the data transfer gets the high priority. It provides two types of data transfer: asynchronous and isochronous. In asynchronous, data transfer can be initiated as a given length of data arrives in a buffer. But, in isochronous data transfer, data flows at a pre-set rate.

Universal Serial Bus (USB)

The Universal Serial Bus (USB) protocol was designed to connect a wide range of peripherals to a computer, including pointing devices, displays, data storage, communication devices and other devices. It standardized the connection of computer peripherals to personal computers, both to communicate and to supply electric power. The original USB 1.0 specification defined data transfer rates of 1.5 Mbit/s for low data rate devices and 12 Mbit/s for high speed devices. The transfer rate for USB 2.0 is 480 Mbit/s while for USB 3.0 it can go up to 5 Gbit/s. USB On-The-Go is the special feature of USB in which two USB devices communicate with each other without requiring a separate USB host. USB uses a tiered star topology, which means some USB devices can serve as connection ports for other USB peripherals. USB hubs and standalone hubs can be used to provide handful of convenient USB ports. USB host controllers manage and control the driver software and bandwidth required by each peripheral connected to the bus.

B. Parallel Protocols

PCI Bus

The Peripheral Component Interconnect (PCI) bus is a high-performance bus for attaching hardware devices in a computer. It is synchronous bus architecture with all data transfers being performed relative to a system clock. The maximum clock rate can go up to 66MHz; however, use of 33MHz is

very common in personal computers. So the transfer rate can vary from 132 to 512 MB/s. PCI implements a 32-bit multiplexed address and Data bus which allows reduced pin count on the PCI connector resulting in lower cost and smaller package size. It supports rigorous auto configuration mechanisms which allow identification of the type of device and the company that produced it. In PCI, any device has the potential to take control of the bus and initiate transactions with any other device making multiple master implementations easier which otherwise had been difficult.

ARM Bus

ARM bus was designed to connect and manage different function blocks in a system on a chip (SoC) designs. It supports 32-bit data transfer and 32-bit addressing and is implemented using synchronous data transfer architecture. The transfer rate is the function of the clock speed used in a particular application. The ARM Advanced Microcontroller Bus Architecture is an open-standard for on chip interconnection.

C. Wireless Protocols

Infrared Data Association (IrDA)

The Infrared Data Association (IrDA) is an international organization that creates and promotes infrared data interconnection standards. It provides specifications for a complete set of protocols for wireless infrared communications. IrDA has been implemented in portable devices like smart phones, laptops, cameras, etc. It is designed to support communication between two devices over point to point infrared at speeds between 9.6 kbps and 4 Mbps. Simplicity and low cost of IrDA hardware makes it an attractive option. Also, line of sight, very low bit error rate and physically secure data transfer are few important features of IrDA. Other wireless technologies with no requirement of direct line of sight have displaced IrDA. However, it is still applicable where interference makes radio based wireless technologies unusable.

Bluetooth

Bluetooth is a wireless technology standard for exchanging data over short distances from fixed and mobile devices. It operates at frequencies between 2402 and 2480 MHz which is the globally unlicensed Industrial, Scientific and Medical (ISM) 2.4 GHz short-range frequent band. Since Bluetooth uses a radio-based link, it does not require line of sight for communication. Bluetooth 4.0 may provide the transfer rate of up to 25Mbps. Bluetooth is a packet based protocol with a master-slave structure and one master may communicates up to maximum of seven slave devices. Low power consumption and short range based communication is the typical feature of Bluetooth.

Permitted transmission power and range of communication depend on the radios class. For class 3 radio, range is up to 1m with max permitted power of about 1mW. The range is about 10m and 2.5mW power is permitted in case of class 2 radios, and class 1 radios have a range of about 100m and 100mW of transmission power. Handsfree headset and wireless speakers are two, out of many, examples using Bluetooth.

IEEE 802.11

IEEE 802.11 is a set of media access control (MAC) and physical layer (PHY) specifications for implementing wireless local area network. IEEE 802.11, often termed as Wi-Fi, has the data transfer rate of around 1 - 2Mbps. IEEE 802.11 has a variety of standards, each with a letter suffix; 802.11a, 802.11b, 802.11g, 802.11n standards are quite common. All these 802.11 Wi-Fi standards operate within the ISM frequency bands. Generally, 2.4 GHz band is common which also makes the chips easier and cheaper to manufacture. The data rate can go up to 54 Mbps with some standard, while few latest standards may support up to 6.75 Gbit/s.

The PHY layer defines the means of transmitting bits over a physical link connecting network nodes. It provides an electrical, mechanical and procedural interface to the transmission medium. Modulation, line coding, synchronization are few functions, out of many, performed by the physical layer. The MAC layer provides the addressing and channel access control mechanism that ensures the communication of several nodes within a shared medium. Each device is assigned a unique serial number which is also known as MAC address. Unique MAC address makes it possible for data packets to be delivered to a destination within a sub-network. Multiple access protocol allows several stations connected to the same physical medium to share it. The most common multiple access protocol is the contention based Carrier Sense Multiple Access with Collision Avoidance (CSMA/CD) protocol.